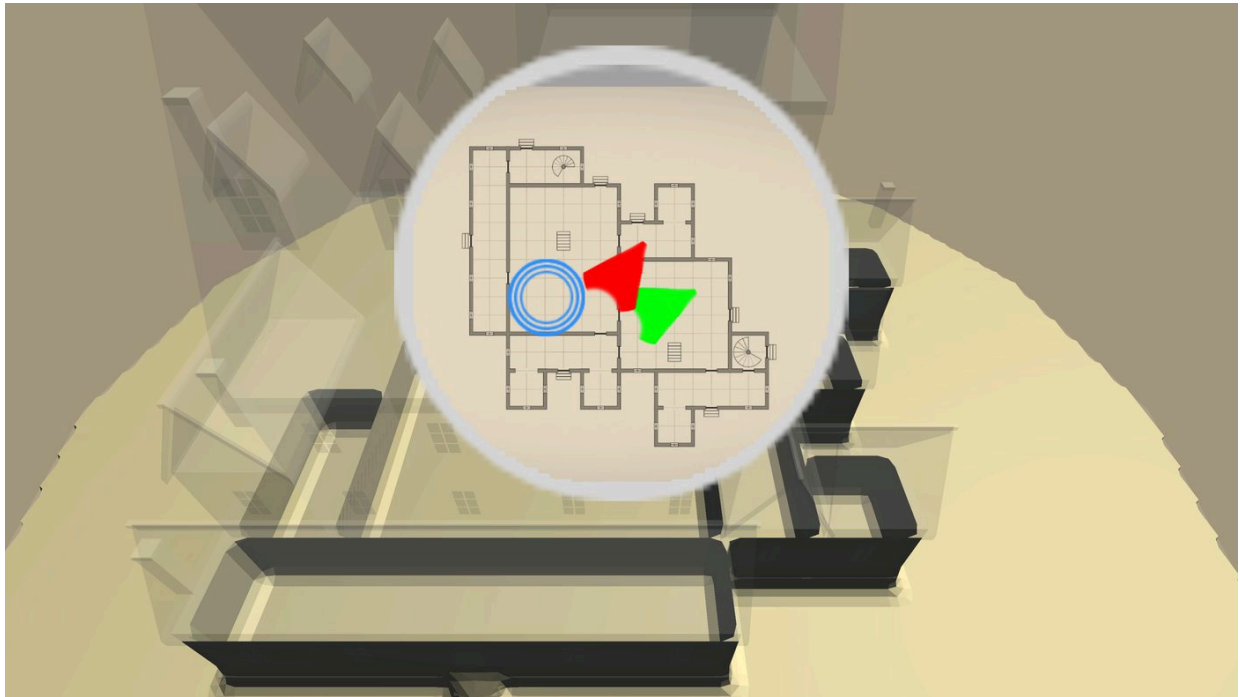


Arikan Simple MiniMap

A lightweight uGUI mini map without extra cameras or RenderTextures



Version 1.2.0 | September 2026 | Bilal Arikan

<https://github.com/bilal-arikan/UnitySimpleMiniMap>

Table of Contents

<u>1. Introduction</u>	3
1.1 Features	3
1.2 Requirements	3
1.3 How It Works	3
<u>2. Installation</u>	4
2.1 From the Asset Store	4
2.2 From a Git URL	4
<u>3. Setup Guide</u>	5
3.1 Step 1: Define the Map Area	5
3.2 Step 2: Prepare the Map Image	5
3.3 Step 3: Build the UI	5
3.4 Step 4: Create an Icon Prefab	6
3.5 Step 5: Configure the MiniMapView	6
3.6 Step 6: Follow Targets from Code	7
<u>4. Features in Detail</u>	8
4.1 Rotation Modes	8
4.2 Zoom	8
4.3 2D Projects (XY Plane)	9
4.4 Icons	9
<u>5. Demo Scene</u>	10
<u>6. Script Reference</u>	11
6.1 MiniMapView	11
6.2 MiniMapBounds	11
6.3 MiniMapMath	12
6.4 Enumerations	12
<u>7. Troubleshooting</u>	13
<u>8. Support</u>	14

1. Introduction

Arikan Simple MiniMap shows your player, enemies, pickups and points of interest on a map image in your UI. The map is a regular uGUI Image and every followed object gets a small UI icon. There is no second camera, no RenderTexture, and there are no extra layers or layer masks, so the mini map costs almost nothing to render and works with every render pipeline.

1.1 Features

- One centered target, usually the player, that stays in the middle of the mini map.
- Any number of additional followed targets.
- Two rotation modes: North Up and Rotate With Target.
- Zoom with configurable limits and step size. Icons keep their size on screen.
- Top-down 3D worlds (XZ plane) and 2D worlds (XY plane).
- Icons that turn with their target or stay upright.
- Icons of destroyed targets are removed automatically.
- Any mask shape: circle, square or a custom sprite.

1.2 Requirements

- Unity 6 (6000.0) or newer.
- Unity UI (uGUI, `com.unity.ugui`). It is installed automatically as a dependency.
- Any render pipeline (Built-in, URP or HDRP). The mini map itself is pure UI. The demo scene uses URP materials, see chapter 5.

1.3 How It Works

MiniMapBounds defines the rectangular world area shown by your map image with two corner Transforms. Every frame, in `LateUpdate`, **MiniMapView** converts the world position of each followed target into a position on the map image and moves its icon there. The centered target is kept in the middle of the mini map by moving the map image instead, and in Rotate With Target mode by rotating it as well.

2. Installation

2.1 From the Asset Store

1. Open **Window > Package Manager** and select **My Assets**.
2. Find **Arikan Simple MiniMap**, click **Download** and then **Import** or **Install**.

2.2 From a Git URL

Add the following line to the `dependencies` of `Packages/manifest.json`:

```
"com.arikan.minimap": "https://github.com/bilal-arikan/UnitySimpleMiniMap.git#1.2.0",
```

3. Setup Guide

The quickest start is to copy `Demo/MiniMap.prefab` and `Demo/MapBounds.prefab` into your scene and replace the map image. The steps below build the same setup from scratch.

3.1 Step 1: Define the Map Area

1. Create an empty GameObject and name it **MapBounds**.
2. Add the **MiniMapBounds** component. Two child corners, **TopRight** and **BottomLeft**, are created automatically.
3. Set **Plane** to **XZ** for top-down 3D games or to **XY** for 2D games.
4. Move the two corners to opposite corners of the area that your map image shows. Their order does not matter.

While MapBounds is selected, the area is drawn as a rectangle gizmo in the Scene view.

3.2 Step 2: Prepare the Map Image

The map image must show exactly the area between the two corners, with world X pointing right and world Z (world Y in 2D) pointing up. An easy way to create it is a top-down orthographic camera that frames the bounds: take a screenshot and crop it to the bounds.

The texture does not have to be square, but its aspect ratio must match the aspect ratio of the bounds. Import it with **Texture Type** set to **Sprite (2D and UI)**.

3.3 Step 3: Build the UI

Create the following hierarchy under a Canvas:

MiniMap	RectTransform + MiniMapView
Mask	Image + Mask, or RectMask2D
Map	Image that shows the map sprite

- **MiniMap** is the root. Place it where the mini map should appear, for example in a corner of the screen.
- **Mask** is the visible window. Use a round sprite with a Mask component for a round mini map, or a RectMask2D for a rectangular one.
- **Map** is the map image and must be a direct child of Mask. Turn off **Preserve Aspect** and **Raycast Target**, and give its rect the same aspect ratio as the bounds, for example 400 x 354 for an area of 51.25 x 45.3 units.

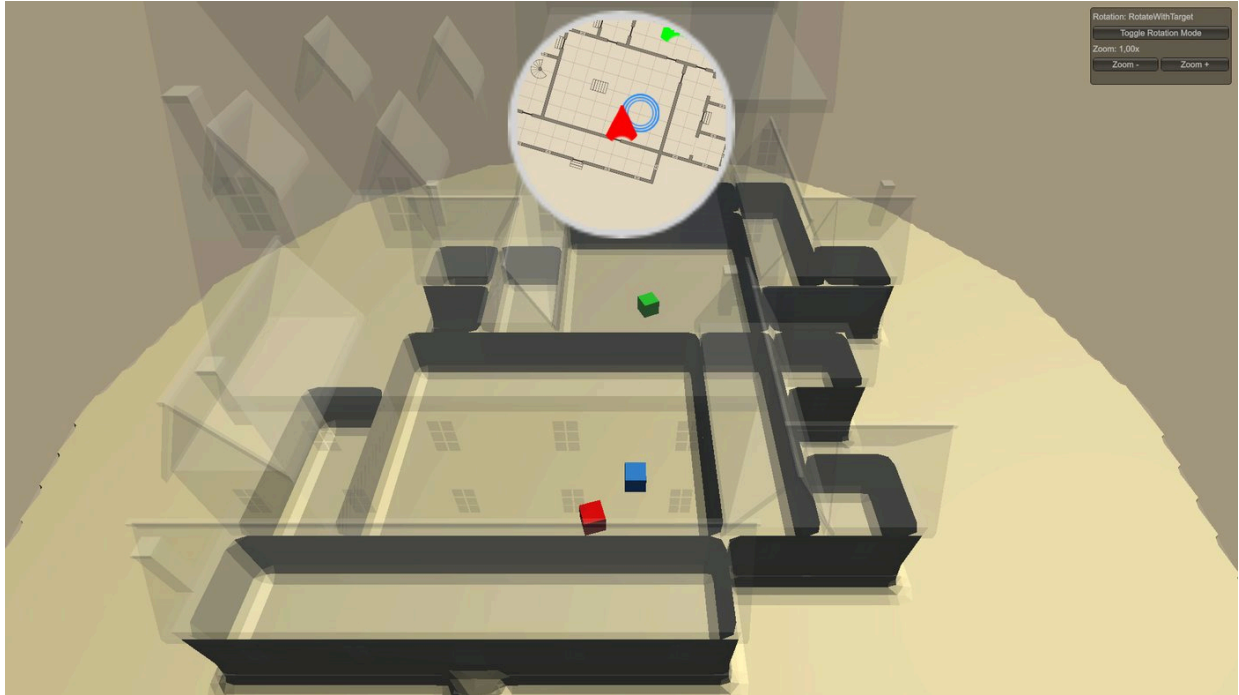


Figure 1: A round mini map at the top of the screen.

3.4 Step 4: Create an Icon Prefab

1. Create a UI Image, for example 40 x 40 pixels, and assign a sprite that points up, such as an arrow.
2. Save it as a prefab. Every followed target gets an instance of this prefab as its icon.

3.5 Step 5: Configure the MiniMapView

Select the MiniMap object and fill in the MiniMapView fields:

Centered Dot Canvas

The Mask object. The centered target is shown at its center.

Other Dot Canvas

The Map object. Icons of the other targets are placed on it.

Default Sprite

Optional. Used when no sprite is passed to Follow or FollowCentered.

Ui Dot Prefab

The icon prefab from step 4.

Mini Map Bounds

The MapBounds object. If it is empty, the first MiniMapBounds in the scene is used.

Rotation Mode

North Up or Rotate With Target, see section 4.1.

Zoom, Min Zoom, Max Zoom, Zoom Step

Zoom settings, see section 4.2.

3.6 Step 6: Follow Targets from Code

Add a script like the following one, assign its fields and press Play. The player stays in the center of the mini map and the other icons move over the map.

```
using Arikan;
using UnityEngine;

public class MiniMapSetup : MonoBehaviour
{
    [SerializeField] private MiniMapView minimap;
    [SerializeField] private Transform player;
    [SerializeField] private Transform enemy;
    [SerializeField] private Transform shop;
    [SerializeField] private Sprite shopIcon;

    private void Start()
    {
        // The player stays in the middle of the mini map.
        minimap.FollowCentered(player).color = Color.red;

        // The enemy icon turns with the enemy.
        minimap.Follow(enemy).color = Color.green;

        // The shop uses its own sprite and stays upright.
        minimap.Follow(shop, shopIcon, rotateWithTarget: false);
    }
}
```

4. Features in Detail

4.1 Rotation Modes

- **Rotate With Target** (default): the map rotates so that the centered target always faces up. The centered icon does not rotate.
- **North Up**: the map keeps world up at the top and the centered icon rotates with the target.

Change the mode in the Inspector or from code:

```
minimap.RotationMode = MiniMapRotationMode.NorthUp;
```



Figure 2: Rotate With Target (left) and North Up (right).

4.2 Zoom

Zoom scales the map around the centered target. Icons keep their size on screen.

```
minimap.Zoom = 2f; // Set the zoom factor directly.  
minimap.ZoomIn(); // Multiply by Zoom Step.  
minimap.ZoomOut(); // Divide by Zoom Step.  
minimap.SetZoomLimits(0.5f, 4f); // Change Min Zoom and Max Zoom.
```

Zoom Step is 1.25 by default. The zoom is always clamped between Min Zoom and Max Zoom.

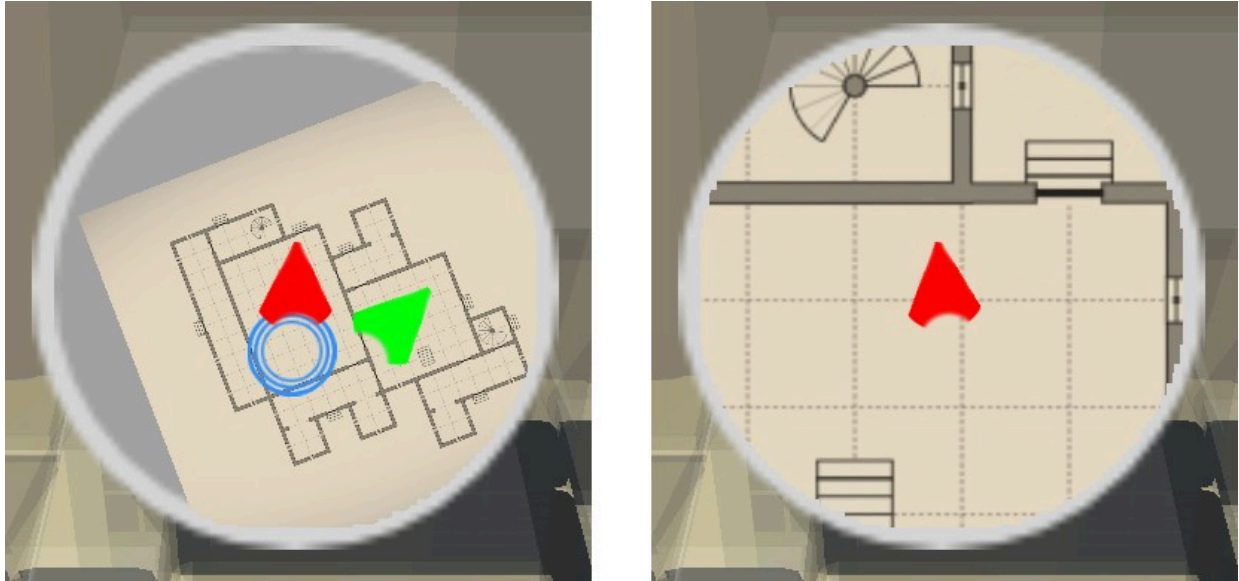


Figure 3: Zoomed out to 0.5x (left) and zoomed in to 3x (right).

4.3 2D Projects (XY Plane)

Set **Plane** on `MiniMapBounds` to **XY** and place the corners in the XY plane. World X points right and world Y points up on the map. Icon rotation follows the up vector of the target, which is its Z rotation in a 2D game, so icon sprites should point up.

4.4 Icons

- **Sprite:** the sprite passed to `Follow` or `FollowCentered` is used first, then the Default Sprite, and finally the sprite of the icon prefab.
- **Color and size:** `Follow` and `FollowCentered` return the Image of the icon. Change its `color`, or its size with `rectTransform.sizeDelta`. Do not change its `localScale`, because it keeps the icon size constant while zooming.
- **Rotation:** icons turn with their target by default. Pass `rotateWithTarget: false` for icons that should stay upright, such as shops or quest markers.
- **Removal:** call `UnfollowTarget(target)` or `ClearTargets()`. Icons of destroyed targets are removed automatically.

5. Demo Scene

Open `Demo/MiniMapDemo.unity` and press Play. The red object walks in a loop and stays in the center of the mini map. The green object is shown with an arrow that turns with it, and the blue object with an upright circle. Use the buttons in the top-right corner to switch the rotation mode and to change the zoom.

The demo scene was made with the Universal Render Pipeline (URP). In the Built-in or HDRP pipeline its materials appear pink, but the mini map works the same way.

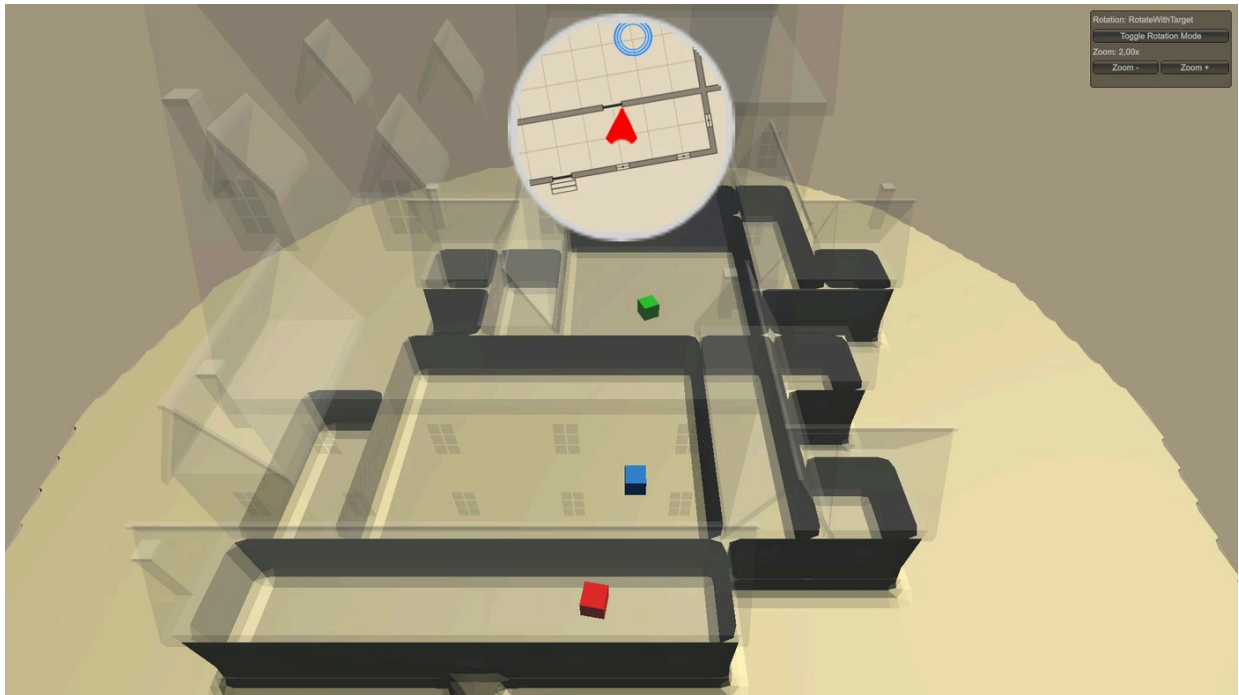


Figure 4: The demo scene with the rotation and zoom controls in the top-right corner.

6. Script Reference

All types are in the `Arikan` namespace.

6.1 MiniMapView

The component that draws the mini map and its icons.

`Image FollowCentered(Transform target, Sprite icon = null)`

Keeps the target at the center of the mini map and returns its icon. Replaces the previous centered target and any existing icon of this target.

`Image Follow(Transform target, Sprite icon = null, bool rotateWithTarget = true)`

Shows the target on the map and returns its icon. Replaces any existing icon of this target.

`void UnfollowTarget(Transform target)`

Removes the icon of the target. Destroyed targets are accepted.

`void ClearTargets()`

Removes all icons, including the centered one.

`bool IsFollowing(Transform target)`

True if the target currently has an icon.

`void ZoomIn() / void ZoomOut()`

Multiplies or divides the zoom by Zoom Step.

`void SetZoomLimits(float min, float max)`

Sets Min Zoom and Max Zoom and clamps the current zoom.

`void Refresh()`

Updates the map and all icons immediately. It is called automatically in LateUpdate.

`MiniMapRotationMode RotationMode`

How the map rotates while a centered target is followed.

`float Zoom`

Current zoom factor, clamped between MinZoom and MaxZoom.

`float MinZoom / float MaxZoom`

Zoom limits (read only).

`Transform CenteredTarget`

The centered target, or null.

`int FollowedTargetCount`

Number of followed targets, not counting the centered one.

Follow and FollowCentered throw an `ArgumentNullException` when the target is null, and an `InvalidOperationException` when a required UI reference or the icon prefab is not assigned.

6.2 MiniMapBounds

Defines the world area shown by the map image.

`MiniMapPlane plane`

World plane shown by the map: XZ for top-down 3D, XY for 2D.

`Transform topRight / Transform bottomLeft`

The two corners of the area. Their order does not matter.

`bool IsValid`

True when both corners exist and span a non-empty area on the selected plane.

`Bounds GetWorldRect()`

Axis-aligned world box between the two corners.

`Rect GetPlaneRect()`

The area between the corners projected onto the map plane.

6.3 MiniMapMath

Static helpers for your own map UI, for example a full-screen world map.

`Vector2 ToPlane(Vector3 world, MiniMapPlane plane)`

Projects a world position onto the map plane.

`Vector2 WorldToMap(Vector3 world, Rect worldRect, Vector2 mapSize, MiniMapPlane plane)`

Converts a world position into a position relative to the center of a map rect of the given size.

`float GetMapAngle(Transform target, MiniMapPlane plane)`

Rotation of the target on the map in degrees, counter-clockwise, where 0 means facing up.

6.4 Enumerations

`MiniMapPlane.XZ / MiniMapPlane.XY`

Top-down 3D worlds / 2D worlds.

`MiniMapRotationMode.NorthUp / MiniMapRotationMode.RotateWithTarget`

Fixed map with a rotating centered icon / rotating map with a fixed centered icon.

7. Troubleshooting

Icons drift away from the picture

The aspect ratio of the Map rect does not match the bounds, Preserve Aspect is enabled, or the image does not cover exactly the bounds area.

Warning: Cannot update the map

A reference is missing, the bounds corners are missing or span an empty area on the selected plane, or the Map rect has no size. The warning names the cause.

The centered target is not in the middle

The Map object must be a direct child of the Mask object.

Icons are too large or too small

Change the size of the icon prefab, or the `rectTransform.sizeDelta` of the returned Image.

Other UI elements update every frame

Put the mini map under its own Canvas so its per-frame updates do not rebuild the rest of your UI.

8. Support

Report bugs and ask questions on GitHub: <https://github.com/bilal-arikan/UnitySimpleMiniMap/issues>

The list of changes in every version is in `CHANGELOG.md` inside the package.